

Incident Report

Secure, Reliable & Scalable Infrastructure for an AI Healthcare Platform

DEVSECOPS • CLOUD ENGINEERING • RELIABILITY • OPERATIONS

Prepared for project submission

N210163 | Naga Mahesh Kona
Rajiv Gandhi University of Knowledge Technologies, Nuzvid
Phone: 9133889049 | Email: n210163@rguktn.ac.in

Version 1.0 | September 2026

Incident: database credentials were missing during startup

What failed

The API service repeatedly restarted during container startup because PostgreSQL authentication was attempted with an empty password. The application expected `DB_NAME`, `DB_USER`, and `DB_PASSWORD` to be defined, but Docker Compose was running without a local `.env` file and the variables were empty.

Impact

- Public API containers failed to initialize and stayed in a restart loop.
- No appointment requests could be accepted while the API was unavailable.
- Monitoring and deployment checks correctly flagged the environment as unhealthy.
- The failure was caused by missing configuration, not by a code bug in the API logic itself.

Detection

- Container logs showed `psycpg2.OperationalError: fe_sendauth: no password supplied`.
- API health checks failed repeatedly on `http://localhost:18080/health`.
- `docker compose ps` showed the API service in a restart state while the database and worker services remained running.

Root cause

The Compose stack referenced environment variables such as `${DB_NAME}`, `${DB_USER}`, and `${DB_PASSWORD}` without default values. On a clean local machine, those variables were unset, resulting in blank credentials and failed PostgreSQL authentication.

Recovery

- Add explicit default values for the local simulation environment in the Compose file.
- Provide a sample `.env.example` file so the expected variables are visible and easy to override.
- Restart the stack cleanly with `docker compose up --build -d` and verify the API health endpoint returns `200`.

Evidence

- API log excerpt: `psycpg2.OperationalError: connection to server at "db" ... fe_sendauth: no password supplied`.
- Container output also showed warnings such as `The "DB_PASSWORD" variable is not set. Defaulting to a blank string`.
- After the fix, the container stack came back healthy and service checks passed.

Prevention

- Always provide defaults for local-only environment values in Compose and keep `.env.example` committed.
- Add a startup validation step in the pipeline so missing secret/environment values fail fast before deployment.
- Keep secrets in `.env` or a real secret manager, never hard-coded in version control.

Additional demonstrated incidents

Worker outage and queue backlog

- **Failure:** The worker container was stopped while the API remained available.
- **Detection:** Redis queue depth increased and the worker health endpoint became unavailable.
- **Recovery:** The worker was restarted; queued work resumed and the queue returned to zero in verification.
- **Prevention:** Keep the worker healthcheck enabled, alert on `up{job="worker"} == 0`, and monitor `worker_queue_depth`.

External EHR outage

- **Failure:** The EHR mock container was stopped while appointment submission remained available.
- **Impact:** New work stayed in Redis and worker attempts were retried rather than silently discarded.
- **Recovery:** The EHR mock was restarted; the worker returned healthy and drained the queue.
- **Prevention:** Alert on `up{job="ehr-mock"} == 0`, retain retry metrics, and use bounded retry/dead-letter behavior in a production implementation.

Recovery and disaster considerations

The local database uses a persistent bind-mounted directory and can also be backed up with `scripts/backup_db.ps1`. `scripts/restore_db.ps1` recreates the simulated appointments table and fails on PostgreSQL errors. Recreating containers restores compute, while restoring the SQL backup restores application state; these are separate recovery actions.